



INF443 FINAL PROJECT

Recreation of Van Gogh's painting of the Church at Auvers-sur-Oise

June 7, 2023

Julien GENZLING - Matthieu DESTRADE



1

INTRODUCTION

We chose to create a 3D scene that recreates Van Gogh's painting of *the Church at Auvers-sur-Oise*. The reason we chose this painting is that every element seems to be in **constant transformation**.

Thus, by recreating the painting in 3D with OpenGL, we are able to capture the subtleties and details that bring Van Gogh's work to life. The different elements of the scene—the tormented sky, the imposing church, and the vibrant trees—are animated to recreate the sense of movement present in the original. Furthermore, the **treatment of color** and the **painter's style** are two essential elements that we wanted to convey as faithfully as possible in the 3D scene, which represented an additional challenge.

In this study, we will describe in detail the process of recreating the painting, focusing on the **technical challenges** encountered and the **artistic choices made**.

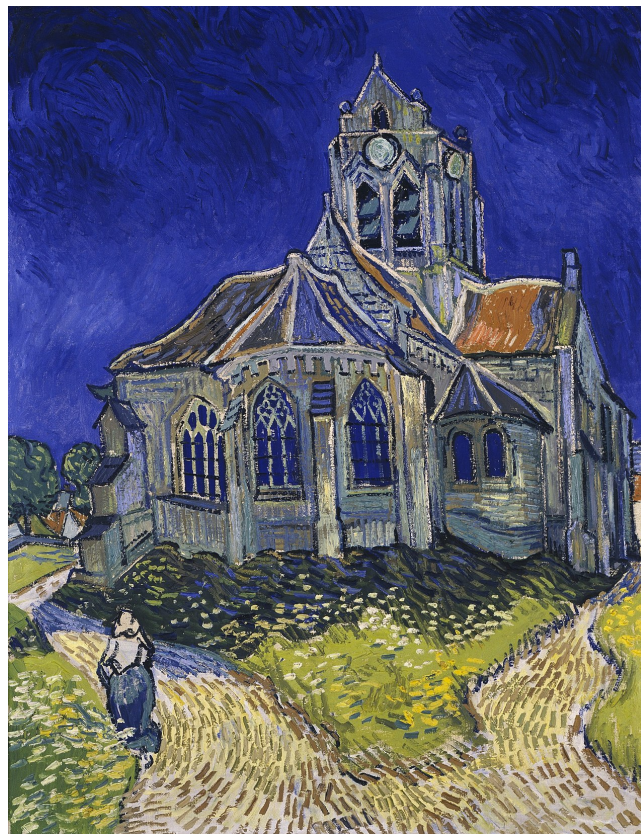


Figure 1: Van Gogh's painting of *the Church at Auvers-sur-Oise*

2 VIEWPOINT OF THE SCENE

Instead of creating a scene that surrounds the camera, we chose to represent the scene within a **unit cube** to reinforce the "3D painting" effect. However, it is possible to enter this cube with the camera. To render this effect, we used **stencil and depth buffers** so that elements are only visible through the cube.

We also added frames around this cube, which likewise make use of **stencil buffer** manipulation so that they are displayed and surround the cube's silhouette regardless of the camera's position. Each part of the frame is associated with a shader that returns a solid color with a slight modification of the color depending on the fragment's position, giving the frame a **shine effect**.

3 ENVIRONMENT

We wanted to evoke Van Gogh's style through the rendering and animation of our environment. To reproduce the impression of **turbulent movement** that his paintings convey, we decided to represent the sky and the ground using **brushstrokes** that would move randomly.

3.1 SIMULATION

Before displaying the brushstrokes, it is necessary to determine their **positions over time**. In order to maintain coherence in the movements, we chose to represent them by the points of a **2D mesh equipped with a physical model** simulating a piece of cloth, that is to say, a grid where each point attracts its neighbors to the right, left, above, and below. More precisely, in our case, the grids have a **toroidal topology** (the extremal points interact with one another, and distance calculations are all done modulo half the size of the grid) so that they form a **repeatable pattern** with no discontinuity. This mesh can then be animated by **noise** and modulated by **specific forces** applied to certain points. Once these 2D positions are obtained, it was necessary to transform them in order to apply them to our 3D space:

- For the **sky**: in order to be visible all around the character, the sky is represented by a **sphere**. The previously computed 2D grid is projected onto it.
- For the **ground**: in order to form hills and relief, we created a function that evaluates the height z of a point on the ground based on its x and y coordinates in the plane. It was then enough to shift each point of the grid vertically according to this function to

obtain the ground. It was also possible to **duplicate the grid** along the x and y axes (which creates no discontinuity since it has a toroidal topology) in order to cover a larger area of terrain without having an overly complex physical simulation to compute.

3.2 PATH

The **path** is a central element of the painting that we sought to recreate faithfully. In order to capture the idea of movement conveyed by the painter, we wanted the path to move like a stream and seem to **flow between the grasses**.

To do this, we first had to code functions producing a list of grid points forming a path from a small number of initially chosen control points, namely a function **generating splines** and another restricting these splines to the grid points. From this list of grid points, it was then possible, by reusing a temporal **interpolation function** used in the lab sessions, to generate points moving between the points of this list over time.

3.3 INSTANCING

Once the set of positions where we wanted to place brushstrokes had been determined, we still needed to display these brushstrokes. To do this, we **instantiated billboards carrying semi-transparent textures** at the points. This involves **sending the shader lists indicating the positions** where the instances should be placed, their rotations (the matrices had to be sent as three lists of vectors), their texture, their color, etc. The use of this **instantiation method**, which adds many more constraints compared to a simple call to the *draw* display function, was necessary for **performance reasons** given the large number of billboards to be displayed. To indicate which texture to display (for example, we wanted to display several different sky textures to avoid repetition effects), and since it seemed impossible to send textures directly to the shader, we associated a **texture map** containing all the images of interest with the billboard to be instantiated. It was then enough to **modify the uv map** used within the shader and shift it depending on which texture we wanted for our billboards.

Depending on the type of object represented, the characteristics of the billboards differ:

- for the **sky**: the billboards are always **oriented toward the camera**, and their **color varies** depending on their position in the grid so as to make the movement effects more visible.
- for the **grass on the ground**: the billboards are **oriented toward the camera**, have a random angle around their **normal** to reduce the texture repetition effect, and rotate on themselves at speeds depending on **Perlin noise** so as to give an impression of wind or waves in addition to the undulating movement of the ground.
- for the **path**: the billboards are **parallel to the ground**, and **oriented in their direction of travel**. The rotation to apply is determined from the spline defining the path and vector calculations.

3.4 SHADERS

Shaders are also used to display the environment. They are used to **darken the grass based on Perlin noise** of the position (in order to reduce tiling and the repetition of a pattern), and make it possible to create **fog** far from the camera to better distinguish the hills.

4 CHURCH, HOUSES, AND TREES

To create the church, we obtained a mesh from the internet. We **modified it in Blender** by deforming its angles, making it more "twisted". The **paintings** were made using the iOS app "Procreate". We reused the code from one of the cgp library's examples to apply these textures to the church as well as to the houses in the scene.

We associate the church with a **specific shader** that enhances the level of its colors and adds a **deformation animation** by multiplying the position in the vertex shader by a matrix that depends on time.

To create the **shadow of the church**, which is an essential element of the painting, we applied color changes in the fragment shader of the grass and the path for fragments located within a zone delimited by affine lines. For the grass, the light green becomes dark but the yellow is not changed, which achieves the same effect as in the painting, and as in the painting, the yellow lines of the path turn blue when they are in the shadow. Finally, we add an effect to smooth the shadow at its edges. The other elements have no shadow, as in the painting.

The trees are textured spheres (using an image created by repeatedly pasting a pattern extracted from the painting so as to create an **isotropic texture**) to which we apply **Perlin noise** in the vertex shader (the code for which was obtained from GitHub <https://gist.github.com/patriciogonzalezvivo/670c22f3966e662d2f83>), which depends on time, making it possible to simulate the random movement of the foliage in the wind.

5 WOMAN

We represented the woman in the painting using a **layered model**, in which a mesh hierarchy animated by **forward kinematics** was combined with a cloth simulated by a **physical model**.

5.1 HIERARCHY

In order to display a character whose limb movements can be animated, we represented the woman by a **hierarchy** comprising a body, two arms, two forearms, and a head. This hierarchy was then animated by varying its degrees of freedom.

5.2 CLOTH SIMULATION

For the character's dress, we simulated a **cylinder**-shaped cloth (the associated mesh being generated by the script) with a physical model very similar to the one used for the environment. In order to obtain a satisfactory shape, the rest lengths of the springs making up the "cloth" **increase toward the bottom of the dress**.

So that the dress does not intersect the woman's legs, we added a **linear potential** pushing the mesh points outward.

To simulate wind on the dress, we added **time-dependent noise** to the velocity of the dress's points.

5.3 MOVEMENT AND ASSEMBLY

The body and the dress then had to be assembled. The main layer is the hierarchy, which **follows the path** (by traveling along the spline defining it) and is subjected to **noise** making it oscillate along the three axes. The secondary layer is the dress, whose **upper ring of points** follows the hierarchy and drags the rest of the dress along with it. The potential preventing intersection with the legs also follows the character. To avoid problems related to initialization and stabilization of the scene in the first moments of computation, the dress is placed on the hierarchy only **after a short delay**.

6

CONCLUSION

In conclusion, over the course of this project we recreated Van Gogh's painting *The Church at Auvers-sur-Oise*. In order to recreate the impression of movement in the painting, we used **several of the methods studied in the lab sessions and beyond**: physical models, splines, hierarchical animation, instancing, shaders...

We left in our virtual world the possibility to **play with different parameters**, in particular those of the physical system: the rigidity of the environmental "cloths", the strength of the perturbations, but also the speed of the paths and whether or not the frame is displayed. However, giving too much importance to the perturbations can lead to **divergences in the calculations**. We were also faced with the fact that when the window displaying the scene is moved, **the scripts pause, but not the timer**, which almost always causes a divergence in all our physical models. Similar problems occur when the framerate is too low. To fix this, **we created a script that only updates the time if the real duration between two frames is not too large**.

We also implemented the possibility of using an **adaptive rendering distance**, which reduces the number of billboards displayed far from the camera (randomly, so as not to create parasitic patterns) in order to **improve performance when necessary**.



Figure 2: Our 3D scene